

QuickJS

Bring JavaScript expressions into DuckDB queries



Install & load

```
INSTALL quickjs FROM community;  
LOAD quickjs;
```

01 At a glance

Evaluate a JavaScript expression, call a function with SQL arguments, or expand an array into rows. Use the embedded QuickJS engine for transformations that benefit from JavaScript control flow, string methods, or JSON reshaping.

SQL arguments → JavaScript → Text / JSON / rows

02 Core SQL surface

```
SELECT quickjs(code)
```

Evaluate JavaScript and return its result as VARCHAR.

```
FROM quickjs(code, ...)
```

Evaluate an array expression; emit one result row per element.

```
quickjs_eval(function, ...)
```

Call a JavaScript function and return JSON.

```
arg0, arg1, ...
```

Access positional arguments in the table form.

```
parsed_arg0, ...
```

Access a parsed copy of JSON strings in the table form.

```
JSON.parse(text)
```

Parse a JSON string argument inside quickjs_eval.

03 Keep in mind

Match the return shape

Scalar quickjs returns VARCHAR, quickjs_eval returns JSON, and FROM quickjs returns rows in a result column. Cast or extract the result explicitly when the surrounding SQL needs a numeric or structured value.

04 Call a function with row values

```
SELECT n, quickjs_eval(  
  'x ⇒ x * x', n  
::INTEGER AS square  
FROM range(1, 4) t(n);  
-- 1, 4, 9
```

Use function parameters for per-row SQL arguments.

05 Unpack a transformed array

```
SELECT result::INTEGER AS value  
FROM quickjs(  
  'parsed_arg0.map(x ⇒ x * arg1)',  
  '[1,2,3]', 3  
);  
-- 3, 6, 9
```

The table form exposes raw and parsed arguments.

06 Reshape a JSON document

```
SELECT quickjs_eval(  
  'text ⇒ { const d = JSON.parse(text); ||  
    'return {user: d.name, active: true}; }',  
  '{"name": "Ada"}'  
);  
-- {"user": "Ada", "active": true}
```

JSON arrives as text here; parse before accessing its fields.

07 In practice

Keep the interpreter focused

JavaScript runs per row and costs more than native SQL. Filter first and reserve it for transformations that need it. This is an embedded language runtime, without browser or Node APIs; use trusted script text.

[Full docs & examples ↗](#) [Source ↗](#)